

Volkart SQL Assignment Write-up

Questions 1-10 | SQL for Modern Data Engineering

Purpose: This document summarizes the SQL answers, the study-note concepts applied, and the reasoning behind each solution.

Question 1. Top 20 Completed Orders By Value

Task: Return the top 20 completed orders by value, with the customer and sales rep behind each order.

SQL Answer

```
USE Volkart;
GO

SELECT TOP 20
    fo.order_id,
    fo.order_date,
    dc.customer_name,
    de.employee_name AS sales_rep_name,
    fo.order_total
FROM dbo.fact_orders AS fo
INNER JOIN dbo.dim_customer AS dc
    ON fo.customer_id = dc.customer_id
INNER JOIN dbo.dim_employee AS de
    ON fo.sales_rep_id = de.employee_id
WHERE fo.order_status = 'Completed'
ORDER BY fo.order_total DESC;
```

Study Notes Concepts Applied

Study Notes concept	Where it appears	Why it matters
Inner join	fact_orders joined to dim_customer and dim_employee	Keeps only rows with matching dimension records.
Fact and dimension model	fact_orders plus customer and employee dimensions	The fact table stores the business event and measures; dimensions add descriptive context.
Filter	WHERE order_status = 'Completed'	Restricts the report to completed orders only.
Top-N query	TOP 20 with ORDER BY order_total DESC	Returns the highest-value orders after sorting.
Execution plan thinking	Join order and physical join choice	The optimizer can choose nested loop, hash join, or merge join depending on indexes and statistics.

How The Query Works

- The query starts from `dbo.fact_orders` because this table contains one row per order and the revenue measure `order_total`.
- It joins `dbo.dim_customer` to turn `customer_id` into `customer_name`.
- It joins `dbo.dim_employee` to turn `sales_rep_id` into `employee_name`, renamed as `sales_rep_name` in the output.

- The WHERE clause keeps only completed orders.
- The ORDER BY sorts completed orders from highest order_total to lowest, and TOP 20 returns only the first 20 rows.

Question 2. Customers Who Never Placed An Order

Task: Marketing wants to re-engage registered customers who never bought.

SQL Answer

```
USE Voltkart;
GO

SELECT
    dc.customer_id,
    dc.customer_name,
    dc.signup_date
FROM dbo.dim_customer AS dc
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.fact_orders AS fo
    WHERE fo.customer_id = dc.customer_id
);
```

Study Notes Concepts Applied

Study Notes concept	Where it appears	Why it matters
Anti join	WHERE NOT EXISTS	Keeps rows from the left table only when no matching row exists in the right table.
Correlated subquery	fo.customer_id = dc.customer_id	Checks orders for the current customer row.
Fact and dimension model	dim_customer against fact_orders	Customers are stored in the dimension; purchases appear in the fact table.
SELECT 1 in EXISTS	SELECT 1 FROM fact_orders	The subquery only needs to prove whether a row exists.

How The Query Works

- The query starts from all customers in dbo.dim_customer.
- For each customer, the NOT EXISTS subquery checks whether an order exists in dbo.fact_orders with the same customer_id.
- If an order exists, the customer is excluded because they have bought at least once.
- If no order exists, the customer is returned as a never-bought customer.

Extra Clarifications

Why anti join: The business question is about absence. INNER JOIN finds customers with matching orders; NOT EXISTS finds customers with no matching orders.

Why not order_total = 0: Customers who never bought normally have no order row at all, not an order row with order_total equal to zero.

Question 3. Top 3 Products Per Category By Completed Revenue

Task: For every category, return the top 3 products by completed revenue.

SQL Answer

```
USE Voltkart;
GO

WITH product_revenue AS (
    SELECT
        dc.category_id,
        dc.category_name,
        dp.product_id,
        dp.product_name,
        SUM(foi.quantity * foi.unit_price) AS total_revenue
    FROM dbo.fact_order_items AS foi
    INNER JOIN dbo.fact_orders AS fo
        ON foi.order_id = fo.order_id
    INNER JOIN dbo.dim_product AS dp
        ON foi.product_id = dp.product_id
    INNER JOIN dbo.dim_category AS dc
        ON dp.category_id = dc.category_id
    WHERE fo.order_status = 'Completed'
    GROUP BY
        dc.category_id,
        dc.category_name,
        dp.product_id,
        dp.product_name
),
ranked_products AS (
    SELECT
        category_id,
        category_name,
        product_id,
        product_name,
        total_revenue,
        RANK() OVER (
            PARTITION BY category_id
            ORDER BY total_revenue DESC
        ) AS revenue_rank
    FROM product_revenue
)
SELECT
    category_id,
    category_name,
    product_id,
    product_name,
    total_revenue,
    revenue_rank
FROM ranked_products
WHERE revenue_rank <= 3
ORDER BY
    category_name,
    revenue_rank;
```

Study Notes Concepts Applied

Study Notes concept	Where it appears	Why it matters
CTE	product_revenue and ranked_products	Breaks a multi-step problem into readable stages.
Inner joins	orders, order items, products, categories	Combines facts with descriptive dimensions.

Study Notes concept	Where it appears	Why it matters
Aggregation	SUM and GROUP BY	Calculates completed revenue per product per category.
Window function	RANK() OVER	Ranks products while keeping product-level rows.
Partitioning	PARTITION BY category_id	Restarts the ranking independently for every category.

How The Query Works

- The first CTE calculates total completed revenue per product by summing quantity times unit_price.
- The joins bring in order status, product name, and category name.
- The WHERE clause keeps only completed orders.
- The second CTE ranks products within each category from highest revenue to lowest revenue.
- The final WHERE clause keeps products whose rank is 1, 2, or 3.

Extra Clarifications

Ties with RANK: If two products have the same total_revenue, they receive the same revenue_rank and the next rank is skipped. Because the filter is revenue_rank <= 3, ties at rank 3 can return more than three products in a category. In this case wasn't necessary utilize DENSE_RANK or ROW_NUMBER.

Question 4. Monthly Completed Revenue, Running Total, And MoM Change

Task: Produce monthly completed revenue with a cumulative running total and month-over-month percent change.

SQL Answer

```
USE Voltkart;
GO

WITH monthly_revenue AS (
    SELECT
        CONVERT(char(7), order_date, 126) AS order_month,
        SUM(order_total) AS monthly_revenue
    FROM dbo.fact_orders
    WHERE order_status = 'Completed'
    GROUP BY
        CONVERT(char(7), order_date, 126)
),
monthly_with_windows AS (
    SELECT
        order_month,
        monthly_revenue,
        SUM(monthly_revenue) OVER (
            ORDER BY order_month
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) AS running_total,
        LAG(monthly_revenue) OVER (
            ORDER BY order_month
        ) AS previous_month_revenue
    FROM monthly_revenue
)
SELECT
    order_month,
```

```

monthly_revenue,
running_total,
CASE
  WHEN previous_month_revenue IS NULL THEN NULL
  WHEN previous_month_revenue = 0 THEN NULL
  ELSE
    ((monthly_revenue - previous_month_revenue) * 100.0)
    / previous_month_revenue
END AS mom_pct_change
FROM monthly_with_windows
ORDER BY order_month;

```

Study Notes Concepts Applied

Study Notes concept	Where it appears	Why it matters
Month bucket	CONVERT(char(7), order_date, 126)	Creates a YYYY-MM grouping key.
CTE	monthly_revenue and monthly_with_windows	Separates monthly aggregation from window calculations.
Aggregation	SUM(order_total) and GROUP BY	Calculates revenue per month.
Window function	SUM(...) OVER	Computes a cumulative running total without collapsing rows.
Window frame	ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	Defines the running total from the first month through the current month.
LAG	LAG(monthly_revenue)	Reads the previous month's revenue for MoM comparison.

How The Query Works

- The first CTE filters to completed orders and groups them by YYYY-MM.
- The second CTE calculates the running total over ordered months.
- The LAG function brings the previous month's monthly_revenue into the current row.
- The final SELECT calculates month-over-month percent change as current minus previous divided by previous, times 100.
- The CASE expression avoids invalid calculations when there is no previous month or the previous month is zero.

Question 5. Customer Quartiles By Lifetime Completed Spend

Task: Split customers into four quartiles by lifetime completed spend and report customer count and average spend per quartile.

SQL Answer

```

USE Volkart;
GO

WITH customer_lifetime_spend AS (
  SELECT
    dc.customer_id,
    dc.customer_name,
    COALESCE(SUM(fo.order_total), 0) AS lifetime_completed_spend
  FROM dbo.dim_customer AS dc

```

```

LEFT JOIN dbo.fact_orders AS fo
  ON dc.customer_id = fo.customer_id
 AND fo.order_status = 'Completed'
GROUP BY
  dc.customer_id,
  dc.customer_name
),
customer_quartiles AS (
  SELECT
    customer_id,
    customer_name,
    lifetime_completed_spend,
    NTILE(4) OVER (
      ORDER BY lifetime_completed_spend DESC
    ) AS spend_quartile
  FROM customer_lifetime_spend
)
SELECT
  spend_quartile,
  COUNT(*) AS customer_count,
  AVG(lifetime_completed_spend) AS avg_lifetime_spend
FROM customer_quartiles
GROUP BY
  spend_quartile
ORDER BY
  spend_quartile;

```

Study Notes Concepts Applied

Study Notes concept	Where it appears	Why it matters
Left join	dim_customer LEFT JOIN fact_orders	Keeps all customers, even customers with no completed orders.
Aggregation	SUM, COUNT, AVG, GROUP BY	Calculates spend per customer and summary metrics per quartile.
COALESCE	COALESCE(SUM(...), 0)	Converts missing spend from NULL to zero.
Window function	NTILE(4) OVER	Splits ordered customers into four groups.
Ordering inside window	ORDER BY lifetime_completed_spend DESC	Places highest spend customers in quartile 1.

How The Query Works

- The first CTE calculates lifetime completed spend per customer.
- The LEFT JOIN keeps customers even if they have no completed orders.
- COALESCE makes customers with no completed orders have spend zero instead of NULL.
- The second CTE sorts customers by spend and assigns a quartile using NTILE(4).
- The final query groups by quartile and returns the number of customers plus the average lifetime spend for each quartile.

Extra Clarifications

Interpretation: Quartile 1 is the highest-value group and quartile 4 is the lowest-value group.

Question 6. Recursive Category Subtree Under Computers

Task: List all categories under Computers at any depth, with depth level and readable path.

SQL Answer

```
USE Voltkart;
GO

WITH category_subtree AS (
    SELECT
        category_id,
        category_name,
        parent_category_id,
        0 AS depth_level,
        CAST(category_name AS varchar(500)) AS category_path
    FROM dbo.dim_category
    WHERE category_name = 'Computers'

    UNION ALL

    SELECT
        child.category_id,
        child.category_name,
        child.parent_category_id,
        parent.depth_level + 1 AS depth_level,
        CAST(parent.category_path + ' > ' + child.category_name AS varchar(500)) AS
category_path
    FROM dbo.dim_category AS child
    INNER JOIN category_subtree AS parent
        ON child.parent_category_id = parent.category_id
)
SELECT
    category_id,
    category_name,
    depth_level,
    category_path
FROM category_subtree
ORDER BY
    category_path
OPTION (MAXRECURSION 0);
```

Study Notes Concepts Applied

Study Notes concept	Where it appears	Why it matters
Recursive CTE	WITH category_subtree AS (... UNION ALL ...)	Walks a hierarchy/tree.
Anchor query	WHERE category_name = 'Computers'	Defines the starting node.
Recursive member	child.parent_category_id = parent.category_id	Finds children of each row already found.
Depth counter	parent.depth_level + 1	Tracks how far each category is from Computers.
Path building	parent.category_path + ' > ' + child.category_name	Builds a readable path through the tree.
MAXRECURSION	OPTION (MAXRECURSION 0)	Removes SQL Server's default recursion limit.

How The Query Works

- The anchor part starts with the Computers category at depth 0.

- The recursive part joins dim_category back to the CTE to find categories whose parent is a category already found.
- Each recursion round moves one level deeper in the tree.
- The depth_level increases by 1 for each child level.
- The category_path appends the child category name to the parent's path.

Extra Clarifications

What OPTION does: OPTION (MAXRECURSION 0) allows the query to traverse the full category tree instead of stopping at SQL Server's default 100 levels.

Question 7. Employee Team Revenue Across The Full Reporting Tree

Task: For every employee, compute completed order value generated by themselves plus everyone reporting under them at any depth.

SQL Answer

```
USE Voltkart;
GO

WITH employee_tree AS (
    SELECT
        employee_id AS manager_employee_id,
        employee_id AS team_member_employee_id
    FROM dbo.dim_employee

    UNION ALL

    SELECT
        et.manager_employee_id,
        child.employee_id AS team_member_employee_id
    FROM employee_tree AS et
    INNER JOIN dbo.dim_employee AS child
        ON child.manager_id = et.team_member_employee_id
),
team_revenue AS (
    SELECT
        et.manager_employee_id AS employee_id,
        SUM(fo.order_total) AS team_total_revenue
    FROM employee_tree AS et
    INNER JOIN dbo.fact_orders AS fo
        ON fo.sales_rep_id = et.team_member_employee_id
    WHERE fo.order_status = 'Completed'
    GROUP BY
        et.manager_employee_id
)
SELECT
    de.employee_id,
    de.employee_name,
    de.role,
    COALESCE(tr.team_total_revenue, 0) AS team_total_revenue
FROM dbo.dim_employee AS de
LEFT JOIN team_revenue AS tr
    ON de.employee_id = tr.employee_id
ORDER BY
    team_total_revenue DESC
OPTION (MAXRECURSION 0);
```


Study Notes Concepts Applied

Study Notes concept	Where it appears	Why it matters
Recursive CTE	employee_tree	Builds each employee's full reporting subtree.
Anchor query	employee_id as manager and team member	Starts every employee's team with themselves.
Recursive member	child.manager_id = et.team_member_employee_id	Finds direct and indirect reports.
Aggregation	SUM(order_total) GROUP BY manager_employee_id	Calculates team revenue per team owner.
Left join	dim_employee LEFT JOIN team_revenue	Keeps employees even when team revenue is zero.

How The Query Works

- The employee_tree CTE creates pairs of team owner and team member.
- In the anchor step, each employee is paired with themselves, so a sales rep's team includes at least themselves.
- The recursive step finds employees who report to the current team member, then repeats for deeper levels.
- team_revenue joins completed orders to each team member and groups the revenue by the original team owner.
- The final SELECT returns every employee with their name, role, and total team revenue.

Extra Clarifications

Why employee_id appears twice: The first employee_id means whose team we are calculating. The second employee_id means the employee inside that team. They are equal in the anchor step, then the recursive step expands the team.

Question 8. Incremental Load From stg_orders_incr Into fact_orders

Task: Use one MERGE to update changed orders and insert new orders from today's staging batch.

SQL Answer

```
USE Voltkart;
GO

DECLARE @before_count int;
DECLARE @after_count int;

DECLARE @updated_sample TABLE (
    order_id int,
    old_order_status varchar(50),
    new_order_status varchar(50),
    old_order_total decimal(18, 2),
    new_order_total decimal(18, 2)
);

SELECT @before_count = COUNT(*)
FROM dbo.fact_orders;
```

```

INSERT INTO @updated_sample (
    order_id,
    old_order_status,
    new_order_status,
    old_order_total,
    new_order_total
)
SELECT TOP 20
    target.order_id,
    target.order_status,
    source.order_status,
    target.order_total,
    source.order_total
FROM dbo.fact_orders AS target
INNER JOIN dbo.stg_orders_incr AS source
    ON target.order_id = source.order_id
WHERE
    ISNULL(target.order_status, '') <> ISNULL(source.order_status, '')
    OR ISNULL(target.order_total, 0) <> ISNULL(source.order_total, 0)
ORDER BY target.order_id;

MERGE dbo.fact_orders AS target
USING dbo.stg_orders_incr AS source
    ON target.order_id = source.order_id

WHEN MATCHED AND (
    ISNULL(target.order_status, '') <> ISNULL(source.order_status, '')
    OR ISNULL(target.order_total, 0) <> ISNULL(source.order_total, 0)
)
THEN UPDATE SET
    target.order_status = source.order_status,
    target.order_total = source.order_total

WHEN NOT MATCHED BY TARGET
THEN INSERT (
    order_id,
    order_date,
    customer_id,
    sales_rep_id,
    order_status,
    order_total
)
VALUES (
    source.order_id,
    source.order_date,
    source.customer_id,
    source.sales_rep_id,
    source.order_status,
    source.order_total
);

SELECT @after_count = COUNT(*)
FROM dbo.fact_orders;

SELECT
    @before_count AS fact_orders_count_before,
    @after_count AS fact_orders_count_after,
    @after_count - @before_count AS rows_inserted;

SELECT
    order_id,
    old_order_status,
    new_order_status,
    old_order_total,
    new_order_total
FROM @updated_sample
ORDER BY order_id;

```

Study Notes Concepts Applied

Study Notes concept	Where it appears	Why it matters
Incremental load	stg_orders_incr into fact_orders	Loads only today's new or changed records, not the full table.
Staging table	dbo.stg_orders_incr	Temporary landing area for the batch.
MERGE / upsert	MERGE dbo.fact_orders	Handles updates and inserts in one statement.
Matched rows	WHEN MATCHED	Existing orders found by order_id.
New rows	WHEN NOT MATCHED BY TARGET	Orders in staging that do not exist yet in fact_orders.
Verification	@before_count, @after_count, @updated_sample	Shows inserted row count and a sample of updated rows.

How The Query Works

- The script first counts rows in fact_orders before the load.
- It captures up to 20 rows that will be updated, storing old values from fact_orders and new values from staging.
- The MERGE matches source and target by order_id.
- If the order exists and status or total changed, the MERGE updates order_status and order_total.
- If the order does not exist in fact_orders, the MERGE inserts it.
- After the MERGE, the script counts rows again and displays the update sample.

Extra Clarifications

What changed means: An order changed when it already exists in fact_orders but the staging value for order_status or order_total is different.

Why TOP 20 exists: TOP 20 is only for a verification sample. It does not limit the MERGE.

Why the sample is captured before MERGE: After the MERGE, the old values are gone from fact_orders, so the script stores old and new values before updating.

ISNULL comparison: ISNULL turns NULL into a comparison value, so SQL Server can detect changes even when one side is NULL.

Question 9. Apply Product Change Feed To dim_product

Task: Use one MERGE to apply operation codes from cdc_product_changes: I inserts, U updates, D deletes.

SQL Answer

```
USE Voltkart;
GO

DECLARE @change_verification TABLE (
    applied_action varchar(10),
    product_id int,
    old_product_name varchar(100),
    new_product_name varchar(100),
    old_unit_price decimal(18, 2),
```

```

        new_unit_price decimal(18, 2)
    );

WITH latest_changes AS (
    SELECT
        change_id,
        product_id,
        operation,
        change_ts,
        product_name,
        category_id,
        unit_price,
        unit_cost,
        launch_date,
        ROW_NUMBER() OVER (
            PARTITION BY product_id
            ORDER BY change_ts DESC, change_id DESC
        ) AS rn
    FROM dbo.cdc_product_changes
)
INSERT INTO @change_verification
SELECT
    CASE
        WHEN lc.operation = 'I' AND dp.product_id IS NULL THEN 'INSERT'
        WHEN lc.operation = 'U' AND dp.product_id IS NOT NULL THEN 'UPDATE'
        WHEN lc.operation = 'D' AND dp.product_id IS NOT NULL THEN 'DELETE'
    END AS applied_action,
    lc.product_id,
    dp.product_name AS old_product_name,
    lc.product_name AS new_product_name,
    dp.unit_price AS old_unit_price,
    lc.unit_price AS new_unit_price
FROM latest_changes AS lc
LEFT JOIN dbo.dim_product AS dp
    ON lc.product_id = dp.product_id
WHERE lc.rn = 1
    AND (
        (lc.operation = 'I' AND dp.product_id IS NULL)
        OR (lc.operation = 'U' AND dp.product_id IS NOT NULL)
        OR (lc.operation = 'D' AND dp.product_id IS NOT NULL)
    );

MERGE dbo.dim_product AS target
USING (
    SELECT
        product_id,
        operation,
        product_name,
        category_id,
        unit_price,
        unit_cost,
        launch_date
    FROM (
        SELECT
            product_id,
            operation,
            product_name,
            category_id,
            unit_price,
            unit_cost,
            launch_date,
            ROW_NUMBER() OVER (
                PARTITION BY product_id
                ORDER BY change_ts DESC, change_id DESC
            ) AS rn
        FROM dbo.cdc_product_changes
    ) AS ranked_changes
    WHERE rn = 1
) AS source
    ON target.product_id = source.product_id

```

```

WHEN MATCHED AND source.operation = 'U'
THEN UPDATE SET
    target.product_name = source.product_name,
    target.category_id = source.category_id,
    target.unit_price = source.unit_price,
    target.unit_cost = source.unit_cost,
    target.launch_date = source.launch_date

WHEN MATCHED AND source.operation = 'D'
THEN DELETE

WHEN NOT MATCHED BY TARGET AND source.operation = 'I'
THEN INSERT (
    product_id,
    product_name,
    category_id,
    unit_price,
    unit_cost,
    launch_date
)
VALUES (
    source.product_id,
    source.product_name,
    source.category_id,
    source.unit_price,
    source.unit_cost,
    source.launch_date
);

SELECT
    applied_action,
    COUNT(*) AS product_count
FROM @change_verification
GROUP BY applied_action;

SELECT
    applied_action,
    product_id,
    old_product_name,
    new_product_name,
    old_unit_price,
    new_unit_price
FROM @change_verification
ORDER BY
    applied_action,
    product_id;

```

Study Notes Concepts Applied

Study Notes concept	Where it appears	Why it matters
CDC / change feed	cdc_product_changes	Source emits row-level changes with operation codes.
MERGE	MERGE dbo.dim_product	Applies inserts, updates, and deletes in one statement.
Dimension sync	dim_product as target	Keeps the product dimension aligned with the source feed.
Operation codes	I, U, D	Drive insert, update, and delete behavior.
Window function	ROW_NUMBER() OVER	Keeps only the latest change per product.
Verification	@change_verification	Stores action-level detail before applying changes.

How The Query Works

- The verification table records which products are expected to be inserted, updated, or deleted.
- latest_changes assigns a row number per product ordered by most recent change_ts and change_id.

- Only `rn = 1` is used so one latest change per product is applied.
- The MERGE matches source changes to `dim_product` by `product_id`.
- If operation is U and the product exists, the product attributes are updated.
- If operation is D and the product exists, the product is deleted.
- If operation is I and the product does not exist, a new product is inserted.
- The final verification queries count and list the applied actions.

Extra Clarifications

Why latest change only: A change feed may contain multiple changes for the same product. `ROW_NUMBER` protects the MERGE by applying only the latest one.

Question 10. Optimize A Slow Report With SARGability, Pre-Aggregation, And Indexes

Task: Rewrite a slow report, add helpful indexes, compare logical reads before and after, and explain why the plan changed.

SQL Answer

```
USE Voltkart;
GO

SET STATISTICS IO, TIME ON;
GO

WITH orders_2024 AS (
    SELECT
        o.customer_id,
        COUNT(*) AS orders_2024
    FROM dbo.fact_orders AS o
    WHERE o.order_date >= '2024-01-01'
        AND o.order_date < '2025-01-01'
    GROUP BY
        o.customer_id
),
customer_lifetime_value AS (
    SELECT
        o.customer_id,
        SUM(oi.line_amount) AS lifetime_value
    FROM dbo.fact_orders AS o
    INNER JOIN dbo.fact_order_items AS oi
        ON o.order_id = oi.order_id
    GROUP BY
        o.customer_id
)
SELECT
    o24.customer_id,
    o24.orders_2024,
    clv.lifetime_value
FROM orders_2024 AS o24
LEFT JOIN customer_lifetime_value AS clv
    ON o24.customer_id = clv.customer_id
ORDER BY
    o24.customer_id;

SET STATISTICS IO, TIME OFF;
GO
```

```
CREATE INDEX IX_fact_orders_order_date_customer
ON dbo.fact_orders (order_date, customer_id)
INCLUDE (order_id);

CREATE INDEX IX_fact_order_items_order_line_amount
ON dbo.fact_order_items (order_id)
INCLUDE (line_amount);
```

Study Notes Concepts Applied

Study Notes concept	Where it appears	Why it matters
Execution plan reading	Comparing original and optimized plans	Shows whether SQL Server scans or seeks, and which joins/aggregates are expensive.
SARGability	Date range instead of YEAR(order_date)	Allows SQL Server to use an index on order_date.
Correlated subquery rewrite	customer_lifetime_value CTE	Calculates lifetime value once per customer instead of expressing it as a per-row lookup.
Pre-aggregation	CTEs grouped by customer_id	Reduces repeated work before the final join.
Covering indexes	CREATE INDEX with INCLUDE	Lets SQL Server read needed columns from narrower nonclustered indexes.
STATISTICS IO, TIME	SET STATISTICS IO, TIME ON	Provides logical reads and timing for before/after comparison.

How The Query Works

- The original query used YEAR(o.order_date) = 2024, which forced SQL Server to compute YEAR() on the column and made the predicate non-SARGable.
- In the original execution plan, SQL Server used table scans on fact_orders and fact_order_items, plus hash joins and hash aggregates.
- The optimized query replaces YEAR() with a date range: order_date >= '2024-01-01' and order_date < '2025-01-01'.
- The orders_2024 CTE counts 2024 orders per customer.
- The customer_lifetime_value CTE pre-aggregates lifetime value once per customer.
- The final SELECT left joins the two aggregated results, preserving customers with 2024 orders.
- The CREATE INDEX statements should be run before the final optimized test, because SQL Server can only choose indexes that already exist when it compiles and executes the query plan.
- The fact_orders index orders the data by order_date and customer_id, helping the 2024 branch seek the date range and group by customer with fewer reads.
- The fact_order_items index is organized by order_id and includes line_amount, helping the join and revenue aggregation without reading unnecessary table columns.
- After adding indexes, the optimized plan uses an index seek on fact_orders for the 2024 date branch.
- The lifetime value branch can still scan indexes because it needs all historical orders, but it scans narrower nonclustered indexes instead of full tables.

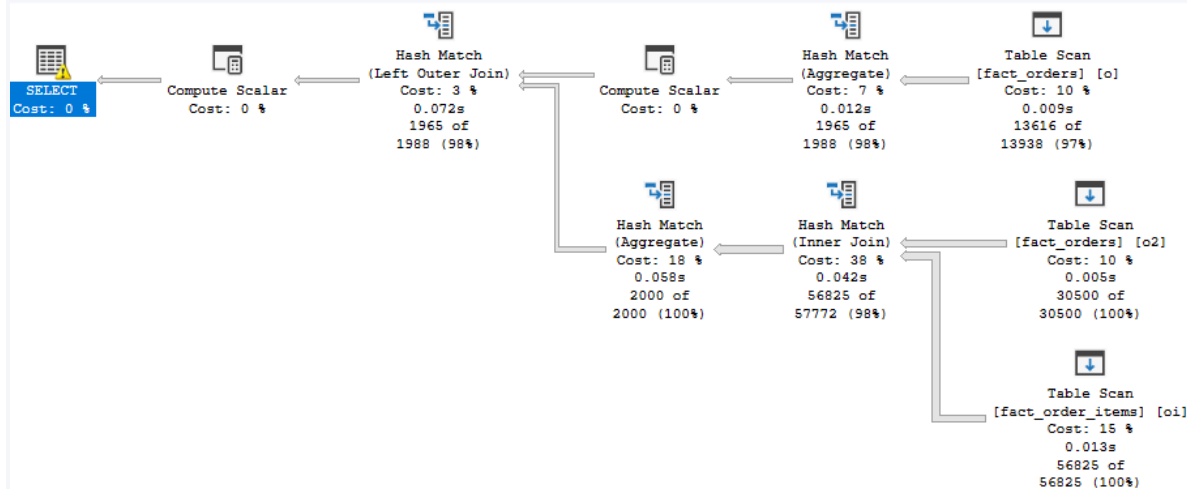
Extra Clarifications

Original plan: The first plan showed table scans on fact_orders and fact_order_items, compute scalar for YEAR(), hash aggregate, hash inner join, and hash left outer join.

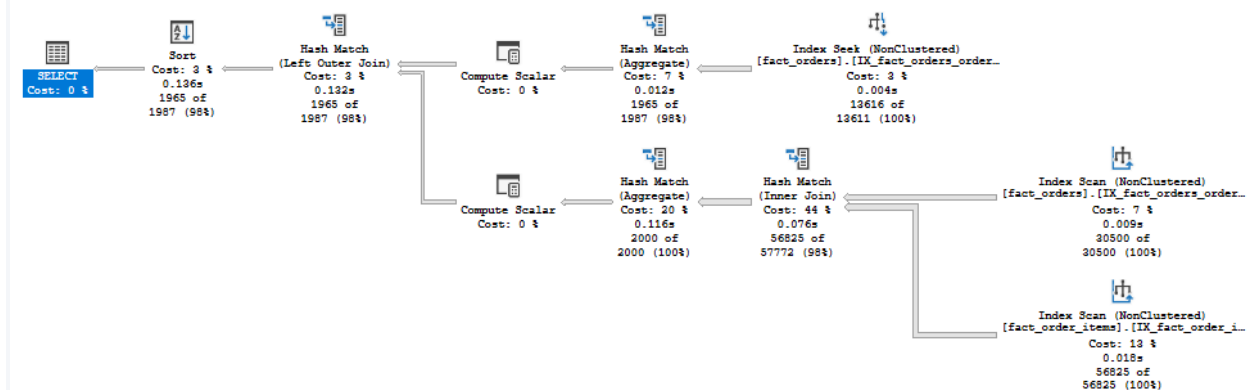
Optimized plan: The optimized plan showed an index seek on the 2024 date-range branch, plus index scans on the lifetime-value branch and hash operations for grouping/joining.

Why time improved: The date branch changed from scanning all orders and computing YEAR() to seeking the relevant date range. Covering indexes reduced pages read, and the lifetime calculation was pre-aggregated once.

Execution plan before optimization:



Execution plan after optimization:



FROM Non-optimidez query:

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:

CPU time = 0 ms, elapsed time = 0 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 13 ms.

SQL Server Execution Times:

CPU time = 0 ms, elapsed time = 0 ms.

SQL Server parse and compile time:

CPU time = 12 ms, elapsed time = 12 ms.

(1965 rows affected)

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'fact_order_items'. Scan count 1, logical reads 267, physical reads 0, page server reads 0, read-ahead reads 267, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'fact_orders'. Scan count 2, logical reads 384, physical reads 0, page server reads 0, read-ahead reads 192, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 row affected)

SQL Server Execution Times:

CPU time = 62 ms, elapsed time = 31789 ms.

Completion time: 2026-06-28T18:30:37.3493521-03:00

From Optimized query:

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 48 ms.

(1965 rows affected)

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'fact_order_items'. Scan count 1, logical reads 186, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'fact_orders'. Scan count 2, logical reads 142, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 row affected)

SQL Server Execution Times:

CPU time = 140 ms, elapsed time = 330 ms.

Completion time: 2026-06-28T18:41:04.2675393-03:00.